Optimizing HQC using Frobenius Additive FFT on a RISC-V-based System-on-Chip

Antonio Ras[1], Antoine Loiseau[1], Mikael Carmona[1], Simon Pontié[1,5], Guénaël Renault[3,4]
Benjamin Smith[3], Emanuele Valea[2]
[1]*CEA-Leti*, Grenoble, France [2]*CEA-List*, Grenoble, France [3]*LIX, INRIA, CNRS, IP Paris*, France
[4]*ANSSI*, Paris, France [5]*Equipe Commune CEA Leti-Mines Saint-Etienne*, Gardanne,
name.surname@cea.fr, guenael.renault@ssi.gouv.fr, smith@lix.polytechnique.fr,

***Abstract*—HQC is a quantum-resistant cryptographic key encapsulation mechanism, recently selected by NIST as a future standard. Polynomial multiplication is one of the most critical operations in HQC. Due to side-channel security concerns, the previously-used sparse-dense method was recently replaced by classical dense-dense multiplication implemented using Karatsuba's algorithm. This change has made polynomial multiplication the primary performance bottleneck, accounting for approximately 95% of the total execution time. This paper presents an alternative polynomial multiplication technique for HQC: the Frobenius Additive Fast Fourier Transform (FAFFT), which provides significant algorithmic-level performance improvements. We also present ANDROMEDA, the first state-of-the-art hardware implementation of FAFFT, and evaluate its performance impact by integrating our solution in a resource-constrained RISC-V-based System-on-Chip scenario. Experimental results show that our solution improves HQC performance by approximately 9.64× and 19.22× across its security levels, making HQC more practical for real-world deployment.**

***Index Terms*—Post-Quantum Cryptography, HQC, Polynomial Multiplication, FAFFT, RISC-V, FPGA, Hardware Acceleration**

I. Introduction

Modern public-key cryptosystems like RSA and ECC depend on the difficulty of problems such as integer factorization and discrete logarithms, which are secure against classical computers. However, Shor's quantum algorithm [1] can efficiently solve these problems, destroying their security. Post-Quantum Cryptography (PQC) [2] aims to develop new cryptosystems based on problems that remain hard even for quantum computers, including problems based on Euclidean lattices and error-correcting codes. In 2016, the National Institute of Standards and Technology (NIST) launched a process to standardize quantum-resistant algorithms for Key Encapsulation Mechanisms (KEM) and Digital Signatures (DS) [3]. Their KEM selection process has now concluded, with ML-KEM (lattice-based) already standardized and HQC [4] (code-based) selected as a future standard, to enhance security and promote cryptographic diversity in the transition to quantum-safe cryptography [5].

HQC is a code-based IND-CCA2 secure KEM whose security relies on the syndrome decoding problem for Quasi-Cyclic codes. Binary polynomial multiplication is currently the major bottleneck for this scheme. Earlier drafts of HQC proposed *sparse-dense* multiplication, exploiting the fixed low Hamming weight of one of the polynomials to skip intermediate operations and improve performance. However, this method was not constant-time, risking side-channel leaks. To mitigate this, the latest HQC specification switched to *dense-dense* multiplication using the 2-way Karatsuba algorithm, which now accounts for about 95% of execution time.

In this paper, we present an alternative method for performing binary polynomial multiplication in HQC. Our approach, called Frobenius Additive Fast Fourier Transform (FAFFT), is a state-of-the-art technique for long binary polynomial multiplication [6], [7]. The FAFFT has already been tested in BIKE [8], a former competitor in the NIST standardization process that was passed over in favour of HQC. We show that applying FAFFT to HQC significantly enhances polynomial multiplication performance on a resource-constrained RISC-V-based platform, achieving speedups between 3.52× and 8.08× across the three HQC security levels. To further boost performance, we introduce ANDROMEDA, a hardware accelerator designed to mitigate the FAFFT's main computational bottlenecks. It leverages a configurable hardware module, the Carry-Free Butterfly, which handles the execution of key processing steps within the multiplication workflow. We integrate our hardware solution into the RISC-V-based platform, using a loosely-coupled approach. ANDROMEDA achieves further speedups of between 2.74× and 2.38× over the FAFFT-based software implementation, resulting in total performance gains between 9.64× and 19.22× over the HQC reference implementation.

To summarize our contributions:

- We integrate the Frobenius Additive FFT (FAFFT) in HQC, showing a large improvement over Karatsuba-based approaches in resource-constrained scenarios;
- We propose ANDROMEDA, a hardware accelerator for the FAFFT;
- We provide a system-level integration of ANDROMEDA in a resource-constrained RISC-V-based System-on-Chip, demonstrating its practical viability and significant performance gains in realistic post-quantum cryptography use cases.

Paper organization: Section II provides background on the HQC cryptosystem and introduces the proposed alternative FAFFT-based multiplication method—with the existing FAFFT state-of-the-art solution. Sections III, IV, and V detail

the three main contributions of this work, along with their corresponding results. Section VI presents our conclusions.

II. Background

We begin with an overview of HQC in Subsection II-A, then describe a generic FFT-based approach for binary multiplication in Subsection II-B. Subsection II-C presents he Frobenius Additive FFT, an optimized FFT-based multiplication method for binary polynomials.

A. HQC

Hamming Quasi-Cyclic (HQC) [4] is an IND-CCA2-secure KEM, recently selected by NIST as future standard. HQC uses two linear codes: a quasi-cyclic random code that underpins the security of the scheme, and a fixed public code with efficient decoding and strong error-correction properties. HQC uses the public code to encode a short random string, from which the KEM session key is derived. The secret key is then used to introduce errors into the encoded string—well beyond the correction threshold of the public code—ensuring that only the entity that generated the cryptographic key pair is able to remove the errors and recover the original message.

TABLE I: HQC Parameter Sets

HQC	n_1	n_2	n	w	$w_{\mathbf{r}} = w_{\mathbf{e}}$	DFR
HQC-128	46	384	17,669	66	75	2^{-128}
HQC-192	56	640	35,851	100	114	2^{-192}
HQC-256	90	640	57,637	131	149	2^{-256}

Table I shows the HQC parameter sets for the three main NIST security levels. The parameters n and w define a ring $\mathcal{R} := \mathbb{F}_2[X]/(X^n + 1)$ with a special subset $\mathcal{R}_w = \{v \in \mathcal{R} \mid \omega(v) = w\}$ where $\omega(\cdot)$ is the Hamming weight. HQC is constructed in two stages: the first defines `HQC.PKE`, an IND-CPA-secure public-key encryption scheme whose security relies on the hardness of the Quasi-Cyclic Syndrome Decoding problem (QCSD) [4]. The second applies the Hofheinz–Hövelmanns–Kiltz (HHK) transform [9] to `HQC.PKE` to derive an IND-CCA2-secure KEM. `HQC.PKE` consists of three algorithms:

- `HQC.PKE.KeyGen`: We uniformly sample a vector $\mathbf{h}$ from $\mathcal{R}$, and secret vectors $\mathbf{x}, \mathbf{y}$ from $\mathcal{R}_w$. The public key is $(\mathbf{h}, s := \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$, and the private key is $(\mathbf{x}, \mathbf{y})$.
- `HQC.PKE.Encrypt`: To encrypt a 32-byte message m, we uniformly sample $\mathbf{r}_1, \mathbf{r}_2 \in \mathcal{R}_{w_r}$, $\mathbf{e} \in \mathcal{R}_{w_e}$, then compute $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2 \in \mathcal{R}$ and $\mathbf{v} = m\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{h} \in \mathcal{R}$, where $m\mathbf{G} = \mathcal{C}.\texttt{Encode}(m)$ is the encoding of m in the public linear code. The ciphertext is $(\mathbf{u}, \mathbf{v})$.
- `HQC.PKE.Decrypt`: To decrypt a ciphertext $(\mathbf{u}, \mathbf{v})$ using the private key $\mathbf{y}$, we compute $m := \mathcal{C}.\texttt{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$, where $\mathcal{C}.\texttt{Decode}(\cdot)$ is the decoding process for the public linear code.

Polynomial multiplication is a critical operation in HQC, appearing in each of the PKE functions. Earlier versions of HQC used special *sparse-dense* multiplications, taking advantage of the fixed and low Hamming weight of one of the operands. This sparse binary polynomial can be represented by the positions of its non-zero coefficients. This representation allows skipping many intermediate operations during the multiplication, improving computational efficiency and overall performance [4], [10]. However, the efficient execution of the sparse-dense approach results in implementations that are not constant-time, which can leak sensitive information through side-channels [4, Section 3.2].

For this reason, sparse-dense multiplication was replaced with a dense-dense multiplication algorithm—2-way Karatsuba—in the latest update of the HQC. Thus, multiplication has become the main bottleneck of HQC, consuming approximately 95% of the total execution time. The Karatsuba approach recursively breaks down the multiplication of large polynomials into smaller and more manageable sub-problems. Once the sub-problems are solved, the intermediate results are combined to obtain the final product. However, for HQC the binary input polynomials range in size from 17,669 to 57,637 bits, so Karatsuba's recursion is deep, with many intermediate polynomials. These require a significant amount of temporary memory and they contribute to increased computation time, further hindering performance.

The need to improve polynomial multiplication in HQC motivates our use of an alternative algorithm: the FAFFT.

B. General FFT-based Binary Multiplication

We briefly recall the Fast Fourier Transform (FFT) based polynomial multiplication technique using *Kronecker segmentation* (see [11] for further detail and background). Suppose we want to compute the product $C(x)$ of two polynomials $A(x) = a_0 + \cdots + a_d x^d$ and $B(x) = b_0 + \cdots + b_d x^d \in \mathbb{F}_2[x]_{\leq d}$, each represented as a bit sequence of length $d + 1$. Filling high-degree coefficients with 0 as required, we can assume $n = 2(d + 1)$ is a power of 2 (note that the coefficients of C fit into a bit sequence of length n). Fix a parameter $0 < w \leq d$ and set $\ell := \lceil d/w \rceil$. Choose an irreducible polynomial g in $\mathbb{F}_2[x]$ of degree $m := 2w$, and let $\mathbb{F}_{2^m} = \mathbb{F}_2[z]/g(z)$. Now, to compute C we proceed as follows:

1) Partition the coefficients of A and B into ℓ blocks of w bits:
$$A(x) = \sum_{i=0}^{\ell-1} A_i(x)\, x^{iw}, \quad A_i(x) := \sum_{j=0}^{w-1} a_{iw+j}\, x^j,$$
$$B(x) = \sum_{i=0}^{\ell-1} B_i(x)\, x^{iw}, \quad B_i(x) := \sum_{j=0}^{w-1} b_{iw+j}\, x^j.$$
2) Now, applying the map $x \mapsto z \in \mathbb{F}_{2^m}$ to each coefficient block, we set
$$A'(y) := \sum_{i=0}^{l-1} a'_i y^i \in \mathbb{F}_{2^m}[y], \quad \text{where } a'_i := A_i(z),$$
$$B'(y) := \sum_{i=0}^{l-1} b'_i y^i \in \mathbb{F}_{2^m}[y], \quad \text{where } b'_i := B_i(z).$$

3) Using the FFT, evaluate $A'(y)$ and $B'(y)$ at 2ℓ points $\alpha_0, \ldots, \alpha_{2\ell-1}$ in $\mathbb{F}_{2^m}[y]_{\leq 2\ell}$ to get

$$(\hat{a}_0, \ldots, \hat{a}_{2\ell-1}) := (A'(\alpha_0), \ldots, A'(\alpha_{2\ell-1})),$$
$$(\hat{b}_0, \ldots, \hat{b}_{2\ell-1}) := (B'(\alpha_0), \ldots, B'(\alpha_{2\ell-1})).$$

4) Perform point-wise multiplication (PWM) on the 2ℓ evaluations:

$$(\hat{c}_0, \ldots, \hat{c}_{2\ell-1}) := (\hat{a}_0 \cdot \hat{b}_0, \ldots, \hat{a}_{2\ell-1} \cdot \hat{b}_{2\ell-1}).$$

5) Use the IFFT to interpolate the $C'(y)$ such that $C'(\alpha_i) = \hat{c}_i$ for $0 \leq i < 2\ell$.
6) Map $C'(y)$ to $C(x) \in \mathbb{F}_2[x]_{\leq n}$ via $z \mapsto x$ and $y \mapsto x^w$, collecting terms and coefficients using the *Interleave and Combine* operation.

C. Frobenius Additive FFT (FAFFT)

We now present the necessary elements for binary polynomial multiplication using the Lin-Cheng-Han Additive FFT (AFFT) [6] and its optimized version based on the Frobenius map [12].

Cantor Basis: The representation of elements of $\mathbb{F}_{2^m}$ as vectors in $\mathbb{F}_2^m$ always depends on a choice of $\mathbb{F}_2$-basis $(\beta_0, \ldots, \beta_{m-1})$ of $\mathbb{F}_{2^m}$. Cantor [13] defined useful bases when m is a power of 2, i.e. $m = 2^{\ell_m}$ for some $\ell_m > 0$. Gao and Mateer [14] give an explicit recursive construction:

$$\beta_0 = 1 \quad \text{and} \quad \beta_i^2 + \beta_i = \beta_{i-1} \text{ for } 0 < i < m. \tag{1}$$

Given a Cantor basis $(\beta_i)_{i=0}^{m-1}$ of $\mathbb{F}_{2^m}$, we define a sequence of subspaces V_k and a corresponding vanishing polynomial $s_k(x)$, for $0 \leq k \leq m$, as follows:

$$V_k := \langle \beta_0, \beta_1, ..., \beta_{k-1} \rangle \quad \text{and } s_k(x) = \prod_{a \in V_k} (x - a) \tag{2}$$

Each $s_k(x)$ vanishing polynomial satisfies:

- *Linearity*: $s_k(x + y) = s_k(x) + s_k(y)$;
- *Two-term form*: $s_k(x) = x^{2^k} + x$ when V_k is a field;
- *Recursivity*: $s_{k+i}(x) = s_k(s_i(x)) = s_i(s_k(x))$.

Fast computation of $\mathbf{s_i}(\alpha)$: We represent elements $\alpha \in \mathbb{F}_{2^m}$ in the Cantor basis using the bijection defined by

$$\phi_\beta : \sum_{j=0}^{m-1} c_j 2^j \longmapsto \sum_{j=0}^{m-1} c_j \beta_j \quad \text{for } c_j \in \{0, 1\}. \tag{3}$$

This suggests a particularly convenient of $\alpha \in \mathbb{F}_{2^m}$ as an unsigned m-bit integer k such that $\phi_\beta(k) = \alpha$. Using this encoding and the respective Cantor properties, we get

$$s_i(\alpha) = s_i(\phi_\beta(k)) = \phi_\beta(k \gg i), \tag{4}$$

where $k \gg i$ denotes the integer k shifted right by i bits. That is: evaluating s_i at α corresponds to computing the image under ϕ_β of the bit-shifted integer $k \gg i$. This makes evaluating s_i extremely fast and convenient.

Basis conversion (BasisCvt): Evaluation and interpolation of a polynomial $f \in \mathbb{F}_2[x]_{<n}$ at $n = 2^{l_n}$ points in $\mathbb{F}_{2^m}$ expressed in a Cantor basis is particularly efficient if f is represented in a particular basis of $\mathbb{F}_2[x]$ called *novelpoly*. Each of its basis elements $X_k(x)$ is constructed as the product of the vanishing polynomials $s_i(x)$ where the i-th bit of k is set:

$$X_k(x) = \prod_{i \geq 0} s_i(x)^{b_i} \quad \text{where } k = \sum b_i 2^i,\ b_i \in \{0, 1\}. \tag{5}$$

We convert polynomials $f(x)$ from the monomial basis to the novelpoly basis by splitting $f(x)$ as $f_0(x) + s_i(x) f_1(x)$, where $s_i(x)$ is a vanishing polynomial with $2^i < \deg(f)$, and repeating recursively. Each division by $s_i(x)$ is done with XOR operations, whose cost depends on the number of terms in s_i. To reduce this cost, we select only the vanishing polynomials $s_{2^i}(x) = x^{2^{2^i}} + x$, which have exactly two terms. This is achieved via variable substitution, expressing f as a power series in $s_{2^i}(x)$. The result ensures both correctness and minimal XOR cost (see [6, §2.4] for more details).

FFT-based operations: Given $f(x) \in \mathbb{F}_{2^m}[x]_{<n}$ converted to $g(X)$ in the novelpoly basis, we can efficiently evaluate f at the set $\hat{\Sigma} := \alpha + V_{\ell_n} = \{\alpha + u \mid u \in V_{\ell_n}\}$ for $\alpha \in \mathbb{F}_{2^m}$ (where $n = 2^{\ell_n}$) using the $\mathtt{FFT}_{\mathrm{LCH}}$ algorithm [6, §2.3.2, Alg. 1]. The general idea of evaluating at all points of V_{ℓ_n} is to divide the subspace in two sets, $V_{\ell_n - 1}$ and $V_{\ell_n - 1} + \beta_{\ell_n - 1} := \{x + \beta_{\ell_n - 1} : x \in V_{\ell_n - 1}\}$.

The algorithm follows a classic divide-and-conquer approach: we rewrite $f(x) = g(X) = p_0(X) + s_i(x) \cdot p_1(X)$, where $s_i(x) = X_{2^i}(x)$ with $2^i = 2^{\ell_n - 1}$, and p_0, p_1 are half-sized parts of g. We then apply the butterfly operation, defined as follows:

$$\begin{aligned} h_0(X) &= p_0(X) + s_i(\alpha) \cdot p_1(X), \\ h_1(X) &= h_0(X) + p_1(X). \end{aligned} \tag{6}$$

The algorithm proceeds recursively on the two half-sized polynomials by calling the $\mathtt{FFT}_{\mathrm{LCH}}$ function again, passing α as the input for $h_0(X)$ and $\alpha + \beta_i$ for $h_1(X)$. The recursion continues until reaching the base case of size 1, where evaluation simply returns the constant coefficient of the polynomial.

The inverse $\mathtt{IFFT}_{\mathrm{LCH}}$ algorithm recursively reconstructs the original polynomial from its evaluations using an inverse butterfly process:

$$\begin{aligned} p_1(X) &= h_0(X) + h_1(X), \\ p_0(X) &= h_0(X) + s_i(\alpha) \cdot p_1(X). \end{aligned} \tag{7}$$

The full procedure consists of ℓ_n layers, each comprising $n/2$ butterfly operations, enabling fast evaluation and interpolation over structured subspaces in $\mathbb{F}_{2^m}$.

New evaluation points: We can reduce the number of evaluation points using the Frobenius map ϕ_2 over $\mathbb{F}_2$, defined by $\phi_2(a) = a^2$ for $a \in \mathbb{F}_{2^m}$. This map satisfies the key property

$$C(\phi_2(a)) = \phi_2(C(a)) \quad \text{for all} \quad C \in \mathbb{F}_2[x]. \tag{8}$$

The starting n evaluation points can be reduced to $n_p = n/m$ using the Frobenius map and, as suggested in [12], by taking

$$\Sigma = \beta_{\ell_{n_p} + m/2} + V_{\ell_{n_p}} \tag{9}$$

where $\ell_{n_p} = \log(n_p) < m/2$ and $\ell_{n_p} + m/2 < m$. Note that the cardinality of the evaluation set is $\#\Sigma = n_p = n/m$.

Evaluating f at Σ corresponds to performing a traditional size-n evaluation of f on the original point-set $\hat{\Sigma} = \alpha + V_{(\ell_{n_p}+\ell_m)}$, avoiding all the redundant operations. To enable this, we define an invertible linear map *encode* which maps $\mathbb{F}_2[x]_{<n} \mapsto \mathbb{F}_{2^m}^{n_p}[x]$ as follows:

$$\sum_{i=0}^{n} f_i x^i \in \mathbb{F}_2[x] \longmapsto \sum_{i=0}^{n_{p-1}} f'_i x^i \in \mathbb{F}_{2^m}^{n_p}[x] \tag{10}$$

where, representing indexes $j := \sum_{i=0}^{\ell_m} j_i 2^i$, the coefficients are computed as

$$f'_i := \sum_{j=0}^{m-1} r_j \cdot f_{j \cdot n_p + i} \quad \text{with} \quad r_j = \prod_{k=1}^{l_m} (\beta_{m/2-k})^{j_k} . \tag{11}$$

The overall process starts by converting the size-n polynomial $f(x)$ into the novelpoly basis. Then, we *encore* the first ℓ_m layers of the size-n $\texttt{FFT}_{\text{LCH}}$ evaluation. After that, the remaining ℓ_{n_p} layers are computed using a standard size-n_p $\texttt{FFT}_{\text{LCH}}$ evaluation over the reduced point set Σ. The last ℓ_m layers of the $\texttt{IFFT}_{\text{LCH}}$ are handled by the *decode* map corresponding to the inverse of the linear map.

The overall Frobenius Additive FFT alternative method for binary polynomial multiplication is reported in Algorithm 1.

Algorithm 1: FAFFT-based Polynomial Multiplication

Input: n: Maximum length of the output polynomial.

Procedure *FAFFT*$(f(x) \in \mathbb{F}_2[x]_{<d}, \Sigma)$
- $(f_0, \ldots, f_{n-1}) \in \mathbb{F}_2^n \leftarrow \texttt{BasisCvt}(f(x))$;
- $(f'_0, \ldots, f'_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow \texttt{Encode}((f_0, f_1, \ldots, f_{n-1}), \Sigma)$;
- $(\hat{f}_0, \ldots, \hat{f}_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow \texttt{FFT}_{\text{LCH}}((f'_0, \ldots, f'_{n_p-1}), \Sigma)$;
- **return** $(\hat{f}_0, \ldots, \hat{f}_{n_p-1})$;

Procedure *FAFFT*$^{-1}(\hat{f}(x) \in \mathbb{F}_{2^m}^{n_p}, \Sigma)$
- $(f'_0, \ldots, f'_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow \texttt{IFFT}_{\text{LCH}}((\hat{f}_0, \ldots, \hat{f}_{n_p-1}), \Sigma)$;
- $(f_0, \ldots, f_{n-1}) \in \mathbb{F}_2^n \leftarrow \texttt{Decode}((f'_0, f'_1, \ldots, f'_{n-1}), \Sigma)$;
- $f(x) \in \mathbb{F}_2[x]_{<n} \leftarrow \texttt{iBasisCvt}(f_0, f_1, \ldots, f_{n-1})$;
- **return** $f(x)$;

Procedure *BitPolyMult*$(a(x), b(x) \in \mathbb{F}_2[x]_{<d})$
- `// Input polynomials evaluation;`
- $(\hat{a}_0, \ldots, \hat{a}_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow \texttt{FAFFT}(a(x), \Sigma)$;
- $(\hat{b}_0, \ldots, \hat{b}_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow \texttt{FAFFT}(b(x), \Sigma)$;
- `// Point-Wise Multiplication (PWM) ;`
- $(\hat{c}_0, \ldots, \hat{c}_{n_p-1}) \in \mathbb{F}_{2^m}^{n_p} \leftarrow (\hat{a}_0 \cdot \hat{b}_0, \ldots, \hat{a}_{n_p-1} \cdot \hat{b}_{n_p-1})$;
- `// Get multiplication result;`
- $c(x) \in \mathbb{F}_2[x]_{<n} \leftarrow \texttt{FAFFT}^{-1}((\hat{c}_0, \ldots, \hat{c}_{n_p-1}), \Sigma)$;
- **return** $c(x)$;

D. Existing FAFFT Software Implementations

Software FAFFT implementations generally fall into two categories based on the technique used to multiply elements of $\mathbb{F}_{2^m}$. For example, [7] proposes a complete software implementation of FAFFT for multiplying long binary polynomials, tailored for Intel Haswell, where field multiplication is performed using the Cantor representation. This implementation optimizes memory access, with SIMD instructions to enhance performance. However, this approach is not directly used for cryptographic applications, specifically for PQC schemes. On the other hand, [8] integrates an efficient FAFFT software implementation into BIKE (a post-quantum KEM passed over by NIST in favor of HQC). This solution, tailored for Intel Haswell and ARM Cortex-M4 processors, multiplies elements using the tower field construction, which gives better performance than the Cantor approach. The proposed solution also uses bit-sliced data representations, improves the encode/decode phases, and provides assembly-based optimizations for Cortex-M4.

As far as we know, this paper is the first to propose implementing HQC based on the FAFFT technique.

III. HQC using Frobenius Additive FFT

This section presents the first software integration of the FAFFT-based multiplication method in HQC. In Subsection III-A, we introduce our FAFFT-based software implementation and further presenting the parameter settings used for integration across all HQC security levels. We finally demonstrate, in Subsection III-B, how the proposed alternative multiplication method shows performance improvements with respect to the reference HQC implementation.

A. FAFFT in Software

We implemented the FAFFT in C, starting from existing state-of-the-art solutions [7], [8]. Our software is intended to provide a clean and portable baseline, so we did not use any SIMD instructions or assembly-level optimizations.

We choose the basic working field

$$\mathbb{F}_{2^{32}} := \mathbb{F}_2[x]/(x^{32} + x^{22} + x^2 + x + 1) .$$

A general scalar multiplication in this field involves a carryless multiplication between the two field elements, followed by a modular reduction using the specified irreducible polynomial, typically performed using the shift-and-add method.

The tower field construction, implemented as a sequence of nested and smaller field extensions, offers advantages in software due to its recursive and modular structure. However, its portability to hardware is more challenging. The layered design requires additional control logic, intermediate storage, and sequential operations, which can increase complexity and limit parallelism. For this reason, in our FAFFT software, we opted for a Cantor representation-based multiplication, which relies solely on XOR and shift operations, enabling simpler, faster, and more parallel hardware solutions.

TABLE II: FAFFT parameters for all HQC levels. n_{HQC} is the n-parameter from the HQC specification, and n denotes the FFT-based operation size in bits.

NIST Level	n_{HQC}	n	m	n_p	l_{n_p}	Σ
HQC-128	17,669	65536	32	2048	11	$\beta_{27} + V_{11}$
HQC-192	35,851	131072	32	4096	12	$\beta_{28} + V_{12}$
HQC-256	57,637					

We integrate our FAFFT implementation across all NIST security levels of HQC, using Algorithm 1 with the parameter

sets in Table II and the evaluation point sets of to Equation (9). We zero-padded polynomials to the next power-of-2 size (required by the FAFFT method). This allows us to use the same FAFFT internal parameters for HQC-192 and HQC-256.

Compute evaluation points: We can significantly improve FAFFT performance by precomputing all $s_i(\alpha)$ constants, along with the evaluation points in Σ for the butterfly structures in Equations (6) and (7). However, for `HQC-192` and `HQC-256`, the number of required constants grows to 4096 32-bit values, imposing considerable storage overhead for the hardware-based approach. To address this issue, we exploit the so-called *four Russians* method [15]. The FAFFT uses fast evaluation of expressions $s_k(\alpha) = \beta_{m/2+i} + s_k(\gamma)$ where $1 \leq i \leq q$ and γ ranges over all the points in V_q. Starting the original $V_{12} = \langle \beta_0, \ldots, \beta_{11} \rangle$, where $q = 12$, we prepare four constant tables: T_0 contains the Cantor basis values $\beta_{m/2+1}, \ldots, \beta_{m/2+q}$, while T_1, T_2, and T_3 contain all of the elements in

$$V_4 = \langle \beta_0, \ldots, \beta_3 \rangle, \langle \beta_4, \ldots, \beta_7 \rangle, \text{and } \langle \beta_8, \ldots, \beta_{11} \rangle,$$

respectively. Splitting the binary vector $\overline{\gamma}$ of $\gamma \in V_{12}$ into 4-bit chunks, i.e. $\overline{\gamma} = \overline{\gamma}_1 + \overline{\gamma}_2 + \overline{\gamma_3}$ with each $\overline{\gamma}_i \in \langle \beta_{4i-4}, \ldots, \beta_{4i-1} \rangle$, we have

$$\gamma = T_1[\overline{\gamma}_1] \oplus T_2[\overline{\gamma}_2] \oplus T_3[\overline{\gamma}_3],$$

using the $\overline{\gamma}_i$ as table indexes.

Overall, this method requires storing 60 32-bit constants and it is fully compatible for `HQC-128`.

B. Performance results

We integrated our FAFFT alternative in the official HQC software implementation (taken from the additional implementation folder of the Round-4 submission package[1]). We benchmarked this across all HQC security levels on an embedded systems platform using the CV32E40P RISC-V-based processor [16] with an Instruction RAM of 256KB and a Data RAM of the same size. The System-on-Chip (SoC) also includes a JTAG module for accessing the memory and the platform registers, and a UART module for serial communication. All components communicate via an OBI bus [17], an open-source interconnect standard for RISC-V systems. The SoC platform has been deployed on the Xilinx Kintex-7 FPGA from Diligent Genesys 2 board `xc7k325t-2ffg900c` [18]. Synthesis and Place&Route are performed using Xilinx Vivado 2019.1 [19]. The code was compiled using the `riscv32-unknown-elf-gcc` compiler with the `-O3` optimization flag.

Since this is the first integration of FAFFT in HQC, we compare our approach solely with the HQC software reference implementation, which relies on a conventional *dense-dense* multiplication strategy. Table III reports cycle counts for key generation, encapsulation, and decapsulation. Our approach improves KEM performance over the baseline implementation by 3.52×, 4,98×, and 8,08× across the three HQC security levels, demonstrating the algorithmic efficiency gains achieved by the FAFFT.

[1]https://pqc-hqc.org/implementation.html

TABLE III: Software execution time (in cycles) for the original HQC and our FAFFT-based variant. Speedups relative to our variant are shown in brackets.

Work	Level	KeyGen	Encaps	Decaps
SW HQC [4]	HQC-128	43,592,464 [**3.57**×]	87,574,395 [**3.52**×]	132,101,254 [**3.47**×]
	HQC-192	132,528,662 [**5.03**×]	265,882,733 [**4.97**×]	399,562,345 [**4.93**×]
	HQC-256	242,546,831 [**8.21**×]	486,241,524 [**8.07**×]	731,405,854 [**7.95**×]
This work (SW)	HQC-128	12,208,482	24,882,981	38,099,269
	HQC-192	26,347,517	53,528,998	81,043,401
	HQC-256	29,545,963	60,238,075	91,987,793

IV. Accelerating FAFFT in Hardware

In this section, we present the first state-of-the-art hardware implementation of the FAFFT-based multiplication method. Subsection IV-A introduces the Carry-Free Butterfly unit, a configurable hardware block designed to accelerate the core FFT-based operations of the alternative FAFFT approach. In subsection IV-B, we propose ANDROMEDA —the first dedicated hardware design aimed at improving the primary FAFFT bottlenecks. In subsections IV-C, we demonstrate the performance advantages of this hardware solution by analyzing a single polynomial multiplication scenario. Finally, in Subsection IV-D we report the FPGA hardware resources of ANDROMEDA.

A. Carry-Free Butterfly (CFB)

We profiled Algorithm 1, including `PWM` step for consistency. We observed that FFT-based operations take up about 79% of the total polynomial multiplication time, and therefore chose to accelerate these steps using a hardware approach.

The FFT-based operations rely on the butterfly structures outlined in Equations 6 and 7. To accelerate them, we introduce the Carry-Free Butterfly (CFB) unit, shown in Figure 1. This is a configurable, pipelined hardware architecture with a 2-clock cycles latency, designed to perform the required FAFFT arithmetic operations in $\mathbb{F}_{2^{32}}$. Figure 2 presents the three specific butterfly configurations that are employed in the computation of the target operations.

The Carry-Free Butterfly unit takes as input two polynomial coefficients and one constant value, and produces two output values. It is composed of a modular carryless multiplier and three carryless adders, implemented using XOR operations. For the multiplication, the hardware implementation includes modular reduction of the multiplier through the *shift-and-add* method, based on the chosen irreducible polynomial. To minimize hardware resources and improve performance, we employ the classical 2-way Karatsuba algorithm for scalar carryless multiplication. This allows us to multiply two 32-bit field elements using three 16-bit multipliers, which we implemented using a Schoolbook-based architecture.

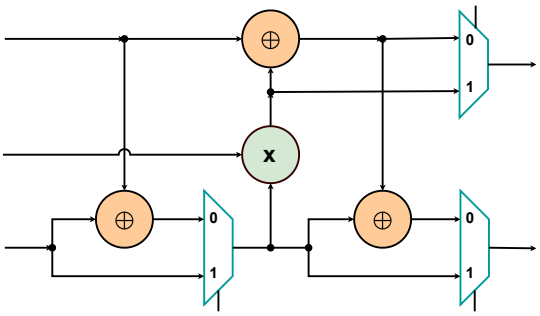
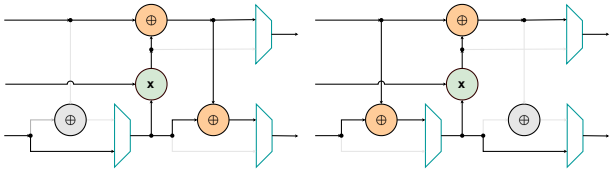
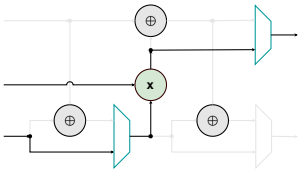


Fig. 1: The configurable Carry-Free Butterfly hardware unit



(a) FFT LCH butterfly (b) IFFT LCH butterfly



(c) PWM butterfly

Fig. 2: Carry-Free Butterfly configuration features for FAFFT multiplication. Note that `PWM` requires only a carryless modular multiplication.

B. ANDROMEDA Design Rationale

To efficiently accelerate FAFFT-based polynomial multiplication in HQC, we leverage the Carry-Free Butterfly unit to perform the $\texttt{FFT}_{\text{LCH}}$, `PWM`, and $\texttt{IFFT}_{\text{LCH}}$ operations within a dedicated hardware accelerator, ANDROMEDA, shown in Figure 3. Our proposal is currently the first proposal for hardware implementation of the FAFFT-based multiplication.

ANDROMEDA performs operations *in-place*, i.e., the memory addresses used to read coefficients are also used to write the corresponding results from the Carry-Free Butterfly units.

Components: ANDROMEDA relies on four internal blocks:

- `Butterfly element` contains two Carry-Free Butterfly units, CFB0 and CFB1, which work independently. They receive a unique control signal to properly configure them as in Figure 2.
- `Memories` store both input polynomials and constant values. The polynomial memories are split into two sets, BRAM_{0-3} and BRAM_{4-7}, which accomodate the two polynomials for the targeted operations.
- `Address generation` calculates the memory indexes of the coefficients consumed by CFB0 and CFB1. Its `Coeffs-indexes` submodule stores index pairs. The `bank-decoder` then determines the corresponding memory addresses and selects the appropriate coefficient instances for processing.
- `Control unit` orchestrate everything, sending control signals to the previous blocks and managing the different polynomial sizes for different HQC security levels.

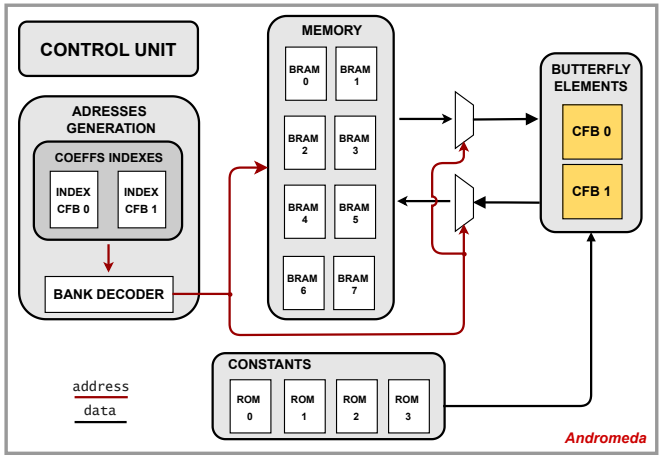


Fig. 3: ANDROMEDA hardware and its internal components

Memories and memory access: The use of two Carry-Free Butterfly units requires reading four polynomial coefficients simultaneously for processing. Since these coefficients are distributed across multiple BRAMs, an inefficient selection of coefficient pairs may lead to performance overhead due to potential memory address conflicts. To prevent this, we adopt the conflict-free memory addressing strategy described in [20], organizing the input polynomials according to this method. While this approach necessitates a more complex bank decoder, it significantly enhances performance by ensuring conflict-free memory accesses and eliminating data dependencies. The two input polynomials are stored in 8 Dual-Port BRAMs, split into two groups of four, one for each polynomial. Each BRAM has 1024 32-bit data words to handle the maximum polynomial size.

Constants generation: We employ the Four Russian method, as explained in Section II-C, by distributing the required constant values across four separate ROMs, each containing 16 entries of 32-bit values. This approach enables the constants to be computed *on-the-fly* by reading the four necessary values in parallel and summing them.

C. Performance results

We implemented Carry-Free Butterfly and ANDROMEDA using SystemVerilog Hardware Description Language (HDL). We simulated, synthetized and implemented them using Vivado 2019.1 design suite on a Xilinx FPGA Artix-7, embedded on a Zybo Z7-20 Diligent board `xc7z020clg400-1` [21]. The critical path delay of ANDROMEDA, imposed by the Carry-Free Butterfly unit, is 8ns; this corresponds to the maximum frequency of 125MHz.

Table IV presents the execution time (in clock cycles) of a single FAFFT polynomial multiplication, showing the

speedup factor between software-only and ANDROMEDA. The hardware-software co-design results exclude the data exchange overhead involved in offloading polynomials from the main memory to the hardware accelerator and vice versa. This overhead will be taken into account in Section V. The results across all HQC security levels, shown in Table IV, demonstrate that ANDROMEDA significantly accelerates the core FAFFT operations: $\texttt{FFT}_{\text{LCH}}$, PWM, and $\texttt{IFFT}_{\text{LCH}}$, achieving speedups of 442×, 385×, and 441×, respectively. Although the remaining FAFFT steps—grouped under the *Others* category—remain unchanged and dominates the hardware-side total, the overall polynomial multiplication execution still benefits from a 4.30× improvement, confirming the impact of offloading the most expensive computations to hardware.

TABLE IV: Cycle counts for FAFFT polynomial multiplication targeting the HQC parameter sets from Table II.

FAFFT Size [bits]	Function	Software	ANDROMEDA	Gain
65536	2×$\texttt{FFT}_{\text{LCH}}$	5,122,527	11,520	*445*×
	1×PWM	430,092	1,102	*390*×
	1×$\texttt{IFFT}_{\text{LCH}}$	2,559,224	5,760	*444*×
	Others	2,518,901	2,518,901	-
	Total	**10,630,744**	**2,537,283**	***4.19***×
131072	2× $\texttt{FFT}_{\text{LCH}}$	11,133,449	25,316	*440*×
	1×PWM	860,172	2,258	*381*×
	1×$\texttt{IFFT}_{\text{LCH}}$	5,562,637	12,658	*439*×
	Others	5,096,923	5,096,923	-
	Total	**22,653,181**	**5,137,155**	***4.41***×

D. Area cost

Table V present the SoC hardware-resources utilization on the Kintex-7 FPGA. As the first-ever hardware implementation of the FAFFT-based multiplication method, our results set a new benchmark in the field. The *Slice Equivalent Cost* (SEC) metric [22] collects and weights various hardware resource costs in a single value. For our Kintex-7 FPGA platform,

$$\text{SEC} := \lfloor \text{LUT}/4 \rfloor + \lfloor \text{FF}/8 \rfloor + 200 \cdot \text{BRAM} + 100 \cdot \text{DSP}\,.$$

Around 14% of ANDROMEDA's slices are dedicated to its two Carry-Free Butterfly units, confirming their pivotal role in accelerating the main computational bottlenecks while maintaining a compact resource utilization.

TABLE V: ANDROMEDA hardware resources utilizations

	LUT	FF	BRAM	DSP	SEC
ANDROMEDA	**2482**	**1158**	**8**	**0**	**2356**
⌊ 1× CFB butterfly	600	127	0	0	166

V. Integration on RISCV-based platform

In this section, we evaluate the performance gains of HQC when using ANDROMEDA in a resource-constrained RISC-V-based system-on-chip scenario.

A. System-on-Chip performance of HQC

We integrated ANDROMEDA as a loosely-coupled accelerator into the already-used RISC-V SoC platform illustrated in Figure 4. We repeated the Synthesis and Place&Route steps using Xilinx Vivado 2019.1, targeting the Kintex-7 FPGA.

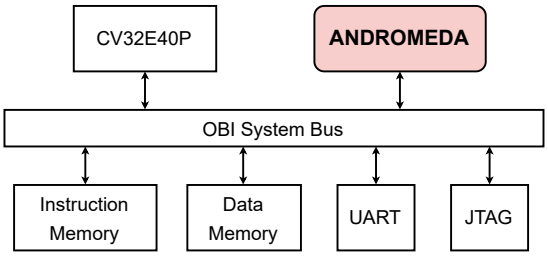


Fig. 4: System-on-Chip (SoC) RISC-V-based platform integrating loosely-coupled ANDROMEDA accelerator

Table VI compares the performance (and relative speed-up) of our ANDROMEDA integration against both the official HQC implementation and our software-only FAFFT-based alternative. For a fair comparison, the co-design was compiled using the `riscv32-unknown-elf-gcc` toolchain with -O3 optimization and executed on the FPGA platform.

Table VI highlights the performance benefits of the proposed FAFFT-based approach across different HQC security levels. The integration of the ANDROMEDA accelerator significantly enhances performance, yielding KEM execution speedups of 9.64×, 13.87× and 19.22× over the original HQC implementation, and providing an additional improvement of 2.74×, 2.79× and 2.38× over our FAFFT-based software variant.

TABLE VI: Cycle counts for official HQC, FAFFT-based software-only, and ANDROMEDA co-design. The relative speed-ups achieved by the co-design are given in brackets.

Work	Level	KeyGen	Encaps	Decaps
SW HQC [4]	HQC-128	43,592,464 [**10.12**×]	87,574,395 [**9.63**×]	132,101,254 [**9.18**×]
	HQC-192	132,528,662 [**14.35**×]	265,882,733 [**13.80**×]	399,562,345 [**13.47**×]
	HQC-256	242,546,831 [**20.27**×]	486,241,524 [**19.39**×]	731,405,854 [**18.01**×]
This work (SW)	HQC-128	12,208,482 [**2.83**×]	24,882,981 [**2.74**×]	38,099,269 [**2.65**×]
	HQC-192	26,347,517 [**2.85**×]	53,528,998 [**2.78**×]	81,043,401 [**2.73**×]
	HQC-256	29,545,963 [**2.47**×]	60,238,075 [**2.40**×]	91,987,793 [**2.27**×]
This work (HW/SW)	HQC-128	4,308,552	9,093,313	14,395,188
	HQC-192	9,233,023	19,273,343	29,659,916
	HQC-256	11,963,005	25,072,159	40,604,459

VI. Conclusion

The future standardization of HQC and its recent multiplication strategy update, driven by security considerations, presents new challenges and opportunities for both software and hardware implementations. In this work, we introduce

the first integration of the alternative Frobenius Additive FFT (FAFFT) multiplication method across all HQC security levels. Our comparison with the latest HQC release demonstrates significant algorithmic improvements with respect to the reference software implementation. Additionally, this technique enables an efficient hardware acceleration strategy, ANDROMEDA, which further boosts HQC performance when integrated into a system-on-chip RISC-V-based scenario, while maintaining minimal hardware resource overhead.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 124–134, IEEE, 1994.

[2] D. J. Bernstein, J. Buchmann, and E. Dahmen, "Introduction to post-quantum cryptography," *Post-Quantum Cryptography*, pp. 1–14, 2009.

[3] L. Chen, S. Jordan, Y.-K. Liu, D. Moody, and R. Peralta, "Report on post-quantum cryptography," Tech. Rep. NISTIR 8105, National Institute of Standards and Technology (NIST), 2016. Accessed: 2024-04-30.

[4] C. Aguilar Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J. Bos, J.-C. Deneuville, A. Dion, P. Gaborit, J. Lacan, E. Persichetti, J.-M. Robert, P. Véron, and G. Zémor, "HQC (Hamming Quasi-Cyclic)," 2024.

[5] G. Alagic *et al.*, "(2025) status report on the fourth round of the nist post-quantum cryptography standardization process," 2025. https://doi.org/10.6028/NIST.IR.8545.

[6] M.-S. Chen, C.-M. Cheng, P.-C. Kuo, W.-D. Li, and B.-Y. Yang, "Faster multiplication for long binary polynomials," *arXiv preprint arXiv:1708.09746*, 2017.

[7] M.-S. Chen, C.-M. Cheng, P.-C. Kuo, W.-D. Li, and B.-Y. Yang, "Multiplying boolean polynomials with Frobenius partitions in additive Fast Fourier Transform," *arXiv preprint arXiv:1803.11301*, 2018.

[8] M.-S. Chen, T. Chou, and M. Krausz, "Optimizing BIKE for the Intel Haswell and ARM Cortex-M4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 97–124, 2021.

[9] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A modular analysis of the Fujisaki–Okamoto transformation," in *Theory of Cryptography Conference*, pp. 341–371, Springer, 2017.

[10] S. Deshpande, C. Xu, M. Nawan, K. Nawaz, and J. Szefer, "Fast and efficient hardware implementation of HQC," in *International Conference on Selected Areas in Cryptography*, pp. 297–321, Springer, 2023.

[11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.

[12] W.-D. Li, M.-S. Chen, P.-C. Kuo, C.-M. Cheng, and B.-Y. Yang, "Frobenius additive Fast Fourier Transform," in *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*, pp. 263–270, 2018.

[13] D. G. Cantor, "On arithmetical algorithms over finite fields," *Journal of Combinatorial Theory, Series A*, vol. 50, no. 2, pp. 285–300, 1989.

[14] S. Gao and T. Mateer, "Additive Fast Fourier Transforms over finite fields," *IEEE Transactions on Information Theory*, vol. 56, no. 12, pp. 6265–6272, 2010.

[15] V. Arlazov, Y. Dinitz, M. Kronrod, and I. Faradzhev, "On economical construction of the transitive closure of an oriented graph," *Dokl. Akad. Nauk SSR*, vol. 194, pp. 487–488, 1970.

[16] OpenHW Group, *CV32E40 User Manual*, 2024. Accessed: 2024-06-11.

[17] OpenHW Group, *Open Bus Interface Protocol*, 2024. https://github.com/openhwgroup/obi, Accessed: 2024-04-07.

[18] I. Digilent, *Genesys 2 FPGA Board Reference Manual*. Digilent, 2017. https://digilent.com/reference/_media/reference/programmable-logic/genesys-2/genesys2_rm.pdf.

[19] I. Xilinx, *Vivado Design Suite User Guide*. Xilinx, 2019. https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0002-vivado-design-hub.html.

[20] J. Mu *et al.*, "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof, and implementation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 5, pp. 1504–1517, 2022.

[21] I. Xilinx, *XC7Z020-1CLG400C: Zynq-7000 All Programmable SoC*, 2023. https://docs.xilinx.com/v/u/en-US/ds190-Zynq-7000-Overview.

[22] M. Li, J. Tian, X. Hu, and Z. Wang, "Reconfigurable and high-efficiency polynomial multiplication accelerator for Crystals-Kyber," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 8, pp. 2540–2551, 2022.